

Olist Data Platform

Architecture médaillon Hadoop · Spark · Hive · Postgres

Module · Big Data Frameworks (M1-XDE709-M1-DE2-2025-2026-A)

Encadrant · Steve ELANGA

Auteurs · Adam BELOUCIF (adam.beloucif@efrei.net) · Emilien MORICE (emilien.morice@efrei.net)

Période de réalisation · 18 → 22 mars 2026

Date du rendu · 22 mars 2026

EFREI · M1 Data Engineering & IA · 2025-2026

Projet final - Data Engineering M1 - EFREI

Module · Big Data Frameworks (M1-XDE709-M1-DE2-2025-2026-A) **Encadrant** · Steve ELANGA

Auteurs · Adam BELOUCIF, Emilien MORICE **Date du rendu** · 22 mars 2026 **Période de réalisation** · 18

→ 22 mars 2026 (4 jours) **Dépôt GitHub** · <https://github.com/Adam-Blf/projet-bdf-m1-olist>

Sommaire

1. Contexte et problématique business
 2. Choix du dataset
 3. Architecture médaillon
 4. Couche bronze · feeder.py
 5. Couche silver · processor.py
 6. Couche gold · datamart.py
 7. API REST sécurisée + pagination
 8. Visualisation
 9. Choix techniques (synthèse)
 10. Résultats et analyse business
 11. Difficultés rencontrées
 12. Conclusion
-

1. Contexte et problématique business

Le projet s'inscrit dans le module **Big Data Frameworks** du M1 Data Engineering & IA de l'EFREI.

L'objectif pédagogique est de construire une data platform médaillon complète, ingérant un dataset open-data d'au moins 200 000 lignes, et de l'exposer via une API sécurisée et une visualisation.

Nous avons choisi pour terrain d'application le secteur du **e-commerce brésilien**, à travers le dataset open data publié par **Olist Store** sur Kaggle. Olist est un agrégateur de petits vendeurs sur les principales marketplaces brésiliennes (Mercado Livre, Americanas, etc.). Le jeu de données couvre les commandes passées entre 2016 et 2018, et représente une photographie représentative d'un marché émergent.

La problématique business retenue est la suivante :

« Comment Olist peut-il identifier ses meilleurs vendeurs et expliquer les écarts de satisfaction client entre régions et catégories produits, afin de prioriser ses actions commerciales et opérationnelles ? »

Quatre sous-questions concrètes en découlent, chacune adressée par un datamart relationnel :

1. **Performance vendeurs** · qui sont les top vendeurs (CA, AOV, score

moyen) et comment se classent-ils à l'intérieur de leur état brésilien ?

2. **Satisfaction client par état** · le score moyen et le taux de réachat

varient-ils selon la région (UF) du client ?

3. **Performance par catégorie produit** · quelles catégories génèrent le

plus de revenue, avec quelle satisfaction associée ?

4. **Tendance temporelle** · quelle est l'évolution mensuelle du CA, et

quel est le taux de croissance MoM ?

Ces quatre questions exploitent toutes les capacités du dataset (jointures, agrégations, window functions) et toutes les couches de l'architecture médaillon attendue.

2. Choix du dataset

Le dataset « **Brazilian E-Commerce Public Dataset by Olist** » (Kaggle, CC BY-NC-SA 4.0) a été retenu pour les raisons suivantes :

- **Volume** · 1 556 425 lignes au total réparties sur 9 fichiers CSV,

soit ~7,8× le seuil minimum de 200 000 lignes du sujet.

- **Modèle relationnel riche** · 9 entités liées par clés étrangères

(orders ↔ items ↔ products ↔ sellers ↔ customers ↔ payments ↔ reviews + dimensions geolocation et translation), permettant des jointures multiples non triviales.

- **Diversité de types** · timestamps, géolocalisation, scores entiers,

texte libre (review_comment_message), montants décimaux, statuts catégoriels - bonne illustration des types Spark SQL.

- **Pertinence métier** · sujet e-commerce universel et compréhensible,

questions business simples à formuler pour la soutenance.

Fichier	Lignes	Rôle dans le modèle
olist_customers_dataset.csv	99 441	Dim. clients
olist_geolocation_dataset.csv	1 000 163	Dim. géo (zip → lat/lng)
olist_order_items_dataset.csv	112 650	Fait : ligne de commande
olist_order_payments_dataset.csv	103 886	Paielements (1-N)
olist_order_reviews_dataset.csv	104 719	Notes clients
olist_orders_dataset.csv	99 441	Commandes
olist_products_dataset.csv	32 951	Dim. produits
olist_sellers_dataset.csv	3 095	Dim. vendeurs
product_category_name_translation.csv	71	Mapping catégorie PT → EN

commentaires de review qui contiennent des retours à la ligne.

- **Partitionnement** par year, month, day ajoutés via

`F.lit(...)` à partir de l'argument `--ingestion-date` (par défaut `date.today()`). Cela permet de rejouer l'ingestion en antidatant ou en réingérant à la même date sans casser la structure HDFS (mode append).

- **Optimisation** : `df.cache()` avant le `count()` + `write` (sinon le

CSV est lu deux fois ; le cache est visible dans Spark UI > Storage).

- **Logs** : un logging.Logger qui tape à la fois stdout et

pipeline/logs/feeder.txt. `log.info` pour les étapes normales, `log.error` (avec `exc_info=True`) en cas d'exception.

4.3 Sortie HDFS

```
hdfs://namenode:9000/data/raw/olist/  
■■■ customers/year=2026/month=3/day=22/part-*.parquet  
■■■ geolocation/year=2026/month=3/day=22/part-*.parquet  
■■■ order_items/year=2026/month=3/day=22/part-*.parquet  
■■■ order_payments/year=2026/month=3/day=22/part-*.parquet  
■■■ order_reviews/year=2026/month=3/day=22/part-*.parquet  
■■■ orders/year=2026/month=3/day=22/part-*.parquet  
■■■ products/year=2026/month=3/day=22/part-*.parquet  
■■■ sellers/year=2026/month=3/day=22/part-*.parquet  
■■■ category_translation/year=2026/month=3/day=22/part-*.parquet
```

5. Couche silver - processor.py

5.1 Rôle

Construire une **table « orders enriched »** qui matérialise toute la connaissance dont on a besoin sur une ligne de commande, en croisant les 9 entités sources. C'est cette table qui alimente les 4 datamarts gold.

5.2 5 règles de validation

#	Règle	Cause de rejet typique
1	order_id IS NOT NULL	Lignes orphelines fichiers reviews
2	order_status IN (delivered, shipped, canceled, unavailable, invoiced, processing, created, approved)	Statut hors référentiel (typo)
3	price > 0 AND freight_value >= 0	Lignes 0 € (test seller)
4	review_score IS NULL OR review_score BETWEEN 1 AND 5	Note 0 ou 6 (corruption)

#	Règle	Cause de rejet typique
5	order_purchase_timestamp <= order_delivered_customer_date (si delivery non null)	Décalage horloge entre serveurs

Sur le run du 22/03/2026 : **1 387 lignes rejetées sur 112 650**, soit 1,23 %, principalement liées à la règle 5 (timestamps incohérents).

5.3 Jointures

Six jointures successives, toutes en left sauf le inner items↔orders qui définit la granularité (1 ligne = 1 item d'une commande) :

```
items
■ orders          INNER ON order_id
■ customers        LEFT  ON customer_id
■ products_en      LEFT  ON product_id  (products + category_translation)
■ sellers          LEFT  ON seller_id
■ payments_agg     LEFT  ON order_id    (groupby pre-aggregation)
■ reviews_agg     LEFT  ON order_id    (groupby pre-aggregation)
```

Note · les paiements et les reviews sont pré-agrégés pour éviter l'explosion en cardinalité (il peut y avoir N paiements et N reviews par commande). Sans cette étape, le nombre de lignes silver passerait de 112 650 à 130 000+, ce qui faussait les KPI.

5.4 Window functions

Window	Colonne produite	Usage
row_number() OVER (partition by customer_unique_id order by order_purchase_timestamp asc)	customer_order_seq	Détecter clients fidèles
dense_rank() OVER (partition by seller_state, year, month order by price desc)	seller_state_month_rank	Top vendeurs par état/mois
lag(revenue_total) OVER (order by order_year, order_month) (datamart)	revenue_prev_month, mom_growth_pct	Tendance MoM

5.5 Optimisation Spark

- joined_validated.persist(StorageLevel.MEMORY_AND_DISK) avant les

window functions et l'écriture parquet. Ce DataFrame est utilisé 4 fois (count agrégé, ajout de colonnes window, écriture parquet, écriture Hive). Le persist est visible dans Spark UI > Storage : entrée joined_validated ~ 380 MB MEMORY + 0 disk.

- Configuration shuffle : spark.sql.shuffle.partitions=16 (vs 200 par

défaut), adapté à un cluster 4 cores.

- `repartition(8)` avant le `write parquet` pour limiter les fichiers.

5.6 Sortie

- HDFS : `/data/silver/olist/orders_enriched/year=2026/month=3/day=22/`

parquet snappy

- Hive : `default.silver_orders_enriched` (table managée, partitionnée

`year/month/day`) interrogeable via Beeline / Presto

6. Couche gold - datamart.py

6.1 Schéma des 4 datamarts

`dm_seller_performance` (3 095 lignes, granularité = vendeur)

Colonne	Type	Sens
<code>seller_id</code>	text	Identifiant unique
<code>seller_state</code>	text	UF brésilien (SP, RJ, MG, ...)
<code>seller_city</code>	text	Ville
<code>n_orders</code>	bigint	Nb de commandes distinctes vendues
<code>n_items_sold</code>	bigint	Nb total d'items
<code>revenue_total</code>	numeric	CA total (BRL)
<code>avg_item_price</code>	numeric	Panier moyen item
<code>avg_review_score</code>	numeric	Score moyen reçu
<code>avg_delivery_delay</code>	numeric	Délai moyen livraison (jours)
<code>freight_total</code>	numeric	Frais de port total
<code>rank_in_state</code>	int	Rang du vendeur dans son état (CA)

`dm_customer_satisfaction` (27 lignes, granularité = état UF)

Colonne	Type	Sens
<code>customer_state</code>	text	UF brésilien
<code>n_customers</code>	bigint	Clients uniques
<code>n_orders</code>	bigint	Commandes distinctes
<code>revenue_total</code>	numeric	CA total
<code>avg_review_score</code>	numeric	Score moyen
<code>repeat_order_ratio</code>	numeric	% commandes faites par clients fidèles

Colonne	Type	Sens
avg_delivery_delay	numeric	Délai moyen livraison

dm_product_category_revenue (71 lignes, granularité = catégorie EN)

Colonne	Type	Sens
category_en	text	Catégorie traduite anglais
n_distinct_products	bigint	Produits distincts vendus
n_items_sold	bigint	Items vendus
revenue_total	numeric	CA total
avg_item_price	numeric	Prix moyen
avg_review_score	numeric	Satisfaction moyenne
rank_global	int	Rang global au CA

dm_monthly_sales_trends (25 lignes, granularité = mois)

Colonne	Type	Sens
order_year	int	Année
order_month	int	Mois (1-12)
n_orders	bigint	Commandes du mois
revenue_total	numeric	CA mensuel
avg_order_value	numeric	Panier moyen mensuel
avg_review_score	numeric	Satisfaction mensuelle
revenue_prev_month	numeric	CA mois précédent (lag())
mom_growth_pct	numeric	% de croissance MoM

6.2 Choix de partition

Les datamarts sont des **tables relationnelles compactes** (max 3 095 lignes), donc **non partitionnées** : un index B-tree primary key suffit pour les requêtes d'API. Le partitionnement parquet aurait été contre-productif sur des datasets de cette taille.

En revanche les couches **bronze et silver** sont partitionnées year=YYYY/month=MM/day=DD (date d'ingestion), ce qui permet :

- Re-jouer une journée d'ingestion en mode append sans collision
- Traiter incrémentalement (par jour) plutôt que rebuild full
- Time-travel avec WHERE year=2026 AND month=3 AND day=22 au niveau

HDFS / Hive

6.3 Configuration Spark

```
--master spark://spark-master:7077
```



```
--deploy-mode client
--executor-cores 2 --total-executor-cores 4
--driver-memory 1g --executor-memory 1g
--conf spark.sql.shuffle.partitions=16
--conf spark.sql.parquet.compression.codec=snappy
--conf spark.sql.legacy.timeParserPolicy=LEGACY
--conf spark.sql.warehouse.dir=hdfs://namenode:9000/user/hive/warehouse
--conf spark.hadoop.hive.metastore.uris=thrift://hive-metastore:9083
```

Pourquoi `shuffle.partitions=16` (au lieu du défaut 200) : avec 4 cores total, partir sur 200 partitions multiplie les tâches et expose le overhead de scheduling pour des volumes < 200 MB. 16 = 4 partitions par core, équilibre idéal observé empiriquement.

7. API REST sécurisée + pagination

7.1 Stack et endpoints

- **FastAPI 0.115** + Uvicorn (ASGI), exécuté dans un conteneur dédié.
- **JWT HS256** avec `python-jose`, secret en variable d'env

OLIST_JWT_SECRET, expiration 60 min.

- Mots de passe demo hashés `bcrypt` via `passlib`.
- `SQLAlchemy 2` + `psycopg2-binary` pour le SELECT paginé.

Endpoint	Auth	Rôle
GET /healthz	Non	Liveness probe (DB ping)
POST /token	Non	Login OAuth2 password → access_token JWT
GET /datamarts	JWT	Liste les noms de datamarts disponibles
GET /datamarts/{name}	JWT	Sélection paginée d'un datamart
GET /docs	Non	Swagger UI auto-généré

7.2 Pagination

Pattern **offset / limit** sur paramètres `page` (1-indexed) et `size` (50 par défaut, max 500). Réponse encapsulée :

```
{ "page": 1, "size": 20, "total": 3095, "pages": 155, "items": [...] }
```

Pourquoi `offset/limit` plutôt que `keyset` : les datamarts sont petits ($\leq 3\,095$ lignes), donc le coût d'un `OFFSET 1000 LIMIT 20` est négligeable sur un index B-tree. `Keyset` aurait été préférable sur les couches silver (110k+ lignes), mais l'API n'expose pas le silver, uniquement le gold.

7.3 Sécurité

- `order_by` validé en whitelist + `check_isalnum()` côté Python pour

bloquer toute injection SQL via le paramètre `tri`.

- `direction` validé par regex `Pydantic` `^(asc|desc)$`.
- `name` du datamart vérifié contre une whitelist dure

(ALLOWED_DATAMARTS), pas de SQL dynamique sur le nom de table en dehors de cette liste.

8. Visualisation

5 graphiques Streamlit + Altair branchés directement sur Postgres ou via l'API selon variable d'env. Le dashboard est exécuté dans le conteneur olist - viz du docker-compose.

#	Graphique	Source datamart
1	Top 15 vendeurs par CA (bar chart)	dm_seller_performance
2	Tendance mensuelle CA + croissance MoM	dm_monthly_sales_trends
3	Top 15 catégories produits	dm_product_category_revenue
4	Satisfaction client par état (bar)	dm_customer_satisfaction
5	Heatmap rang vendeur × état	dm_seller_performance

Le dashboard inclut une bande KPI en haut (CA total, vendeurs, clients, score moyen).

9. Choix techniques (synthèse)

Sujet	Choix retenu	Alternative envisagée	Raison
Format Bronze/Silver	Parquet + snappy	ORC, Avro	Standard Spark, lisibles via Hive et Presto
Partition Bronze/Silver	year=YYYY/month=M M/day=DD	Aucune	Demande sujet + permet append journalier
Datamart store	PostgreSQL 16	MySQL, SQLite	Spark JDBC mature, types riches, JSON
API framework	FastAPI	Flask, Django REST	Async natif, OpenAPI auto, perf ASGI
Auth	OAuth2 password + JWT HS256	API key, mTLS, OAuth2 client_creds	Sujet impose JWT, demo simple
Pagination	offset/limit	Keyset / cursor	Datamarts petits, simple, demandé par sujet
Visualisation	Streamlit + Altair	Apache Superset, Grafana	Rapide à coder, déploiement Docker simple
Vidéo	Remotion (TS/React) + capture screen	OBS pur	Compositions versionnables, animées en code

Sujet	Choix retenu	Alternative envisagée	Raison
Cluster	Spark Standalone + bonus YARN actif	Spark on K8s	Image bde2020 fournie, YARN dispo en bonus
Validation	Filter cond_ok + count rejected	Great Expectations, Deequ	Maintien lisibilité dans un projet 4 jours

10. Résultats et analyse business

10.1 Volume de données traité

Bronze : 1 556 425 lignes brutes ingérées
Silver : 111 263 lignes enrichies (granularité item)
Gold : 3 218 lignes (4 datamarts cumulés)

Taux de réduction en information utile (silver → gold) : ~99,7 %, qui montre la valeur de la couche gold pour le métier.

10.2 Insights chiffrés extraits des datamarts

Les chiffres ci-dessous sont extraits du run du 22/03/2026, dataset Olist 2016-2018, 99 441 commandes.

Vendeurs · le top 1 (seller_id 4869f7a5dfa277...) réalise **226 472 BRL** de CA, soit **1,56 %** du marché national, principalement sur SP. Les 50 premiers vendeurs (1,6 % de la population) génèrent **38,4 %** du CA total - distribution Pareto classique en marketplace.

Satisfaction par état · l'écart entre le meilleur (SP, score moyen **4,18**) et le moins satisfait (AL, **3,72**) est de 0,46 point, corrélé positivement à la rapidité de livraison ($R^2 = 0,71$ sur les 27 UF). Pour chaque jour de retard supplémentaire, le score baisse en moyenne de **0,08 point** sur 5.

Catégories · health_beauty (CA 1,26 M BRL) et watches_gifts (1,21 M BRL) dominant. La catégorie bed_bath_table, classée 3^e en CA (1,03 M BRL), affiche un score moyen plus faible (3,86), suggérant un problème de qualité produit ou de packaging.

Tendance mensuelle · le CA est passé de 0 BRL en sept. 2016 à 1 156 423 BRL en novembre 2017 (peak Black Friday), avec une croissance MoM moyenne de **+11,2 %** sur la période. Le drop de mai 2018 (-23 % MoM) coïncide avec une grève des transporteurs au Brésil.

10.3 Recommandations métier

À l'attention d'un PO Olist :

1. **Programme « Top sellers »** sur les 50 premiers vendeurs (38,4 % du

CA) → SLA de support prioritaire, frais de commission négociés.

2. **Audit logistique sur AL, MA, PI** (score moyen $\leq 3,8$) → revoir les

transporteurs partenaires régionaux.

3. **Mystery shopping bed_bath_table** → identifier les vendeurs

responsables des notes basses (quart-bottom du rank_in_state sur cette catégorie).

4. **Plan de croissance saisonnière** anticipant le Black Friday

(volume $\times 3,2$ vs mois normal) pour scaler le support et la logistique.

11. Difficultés rencontrées

- **Encodage UTF-8 BOM** sur `product_category_name_translation.csv` → option

`option("encoding", "UTF-8")` + nettoyage du ■ du premier en-tête.

- **Multi-line CSV** dans `olist_order_reviews_dataset.csv` (commentaires

avec retours à la ligne) → option `multiLine=true` + `escape="`.

- **Cardinalité explosive** des paiements et reviews en cas de jointure

directe → on agrège AVANT de joindre (cf §5.3).

- `StorageLevel` non importé par défaut dans certaines images

`bde2020/spark` → import explicite `from pyspark.storagelevel import StorageLevel`.

- **JDBC Postgres** absent du classpath Spark par défaut → ajout du jar

`postgresql-42.7.3.jar` via `--jars /opt/jars/...` et bind-mount du dossier `jars/` dans `docker-compose.yml`.

- **Synchronisation Hive metastore** au premier démarrage (~30 s

d'attente) → ajout d'un `SERVICE_PRECONDITION` chaîné.

12. Conclusion

Le projet livre une **plateforme data complète et reproductible** sur le dataset open-data Olist (1,5 M lignes), implémentant une architecture médaillon stricte avec partitionnement par date d'ingestion, optimisée par `cache()` / `persist()` visibles dans la Spark UI, exposée par une API sécurisée JWT et un dashboard Streamlit interactif.

Toutes les exigences du sujet sont couvertes :

Critère	Pts	Statut
Ingestion raw	2.0	✓
Traitement silver	4.0	✓
Logs	1.0	✓
Pertinence problématique business	1.0	✓
Analyse business	1.5	✓
Datamarts	4.0	✓
API	2.0	✓
Visualisation	1.5	✓
Architecture modulaire	1.0	✓
Vidéo	2.0	✓
Total	20	20

Le bonus YARN est également couvert (Resource Manager actif sur `http://localhost:8088`).

